Name : Utsav Golani
Roll No : 2401032
Batch : T12
Subject : ASD&D

# EXPERIMENT – 09

**Aim:** To provision cloud infrastructure using Terraform.

LO4, LO5 and LO6 mapped:

**LO4:**: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

**LO5:** "Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios."

**LO6**: Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.

**Theory:**

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp. It is used to automate the creation, configuration, and management of cloud infrastructure. Terraform allows users to define infrastructure resources such as virtual machines, networks, storage, and databases using configuration files instead of manually creating them.
Terraform uses HashiCorp Configuration Language (HCL) to describe the required infrastructure. It supports various cloud platforms such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP).

**1. Infrastructure as Code (IaC)**

Infrastructure as Code is a method of managing and provisioning cloud resources through code rather than manual configuration. It helps maintain consistency, reduces human errors, and improves automation.

**2. Working of Terraform**

Terraform follows a simple workflow:

Name : Utsav Golani

Roll No : 2401032

Batch : T12

Subject : ASD&D

1. Write: Create Terraform configuration files (.tf) to define the required cloud resources.
2. Initialize: Use terraform init to initialize the working directory and download necessary providers.
3. Plan: Use terraform plan to preview the infrastructure changes.
4. Apply: Use terraform apply to create or update cloud resources.
5. Destroy: Use terraform destroy to remove resources when they are no longer needed.

**3. Important Components of Terraform**

- Providers: Allow Terraform to communicate with cloud platforms such as AWS, Azure, and GCP.
- Resources: Represent infrastructure components such as EC2 instances, storage buckets, and networks.
- State File: Stores information about the infrastructure managed by Terraform.
- Variables: Make Terraform configurations flexible and reusable.
- Modules: Organize and reuse Terraform configuration code.

**4. Advantages of Terraform**

1. Automation: Automates infrastructure provisioning and management.
2. Consistency: Ensures infrastructure is created using standardized configurations.
3. Multi-Cloud Support: Supports multiple cloud service providers. 4. Scalability: Makes it easier to manage large cloud environments.
5. Version Control: Infrastructure configuration files can be maintained using Git.
6. Cost Management: Helps manage resources efficiently and remove unnecessary infrastructure.

**5. Applications of Terraform**

- Creating AWS EC2 instances and S3 buckets.
- Configuring virtual networks and security groups.
- Deploying cloud-based applications.
- Managing infrastructure across multiple cloud platforms.
- Automating development and testing environments.
-

**6. Terraform Initialization**

The first important Terraform command is:

terraform init

Name : Mohit Hotwani

Roll No : 2401037

Batch : T12

Subject : ASD&D

It initializes the working directory and downloads the required provider plugins.

For this experiment, Terraform downloads the **HashiCorp AWS provider**.

Name : Utsav Golani

Roll No : 2401032

Batch : T12

Subject : ASD&D

Successful initialization prepares the directory for further Terraform operations.

**7. Terraform Validate**

The command:

terraform validate checks whether the Terraform configuration is syntactically and

structurally correct.

If the configuration is valid, Terraform displays:

Success! The configuration is valid.

This allows configuration errors to be detected before infrastructure is created.

**8. Terraform Plan**

The command:

terraform plan creates an execution plan showing the changes Terraform

intends to make.

For this experiment, the plan showed:

Plan: 1 to add, 0 to change, 0 to destroy.

This means Terraform identified one EC2 resource that needed to be created.

The plan provides an opportunity to review the proposed infrastructure changes before applying them.

**9. Terraform Apply**

The command: terraform apply applies the configuration and creates the

resources defined in the Terraform files.

After approval, Terraform created the EC2 instance and displayed information such as:

1. Instance ID

Name : Utsav Golani

Roll No : 2401032

Batch : T12

Subject : ASD&D

2. Instance type
3. Public IP address

The successful execution produced:

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

**10. Terraform Output**

The command: terraform output displays the

output values defined in outputs.tf.

In this experiment, it displayed:

instance_id

instance_type

public_ip

This provides a convenient way to retrieve important information about the provisioned infrastructure.

**11. Terraform Destroy**

After completing the experiment, infrastructure should be removed using:

terraform destroy

This removes the resources managed by Terraform.

In this experiment, the EC2 instance was successfully deleted, resulting in:

Destroy complete! Resources: 1 destroyed.

The lab instructions specifically require students to perform terraform destroy after the experiment.

**Terraform Workflow**

The overall workflow followed in this experiment is:

Create Terraform Configuration

↓

terraform init

Name : Utsav Golani

Roll No : 2401032

Batch : T12

Subject : ASD&D

↓

terraform validate

↓

terraform plan

↓

terraform apply

↓

EC2 Instance Created

↓

terraform output

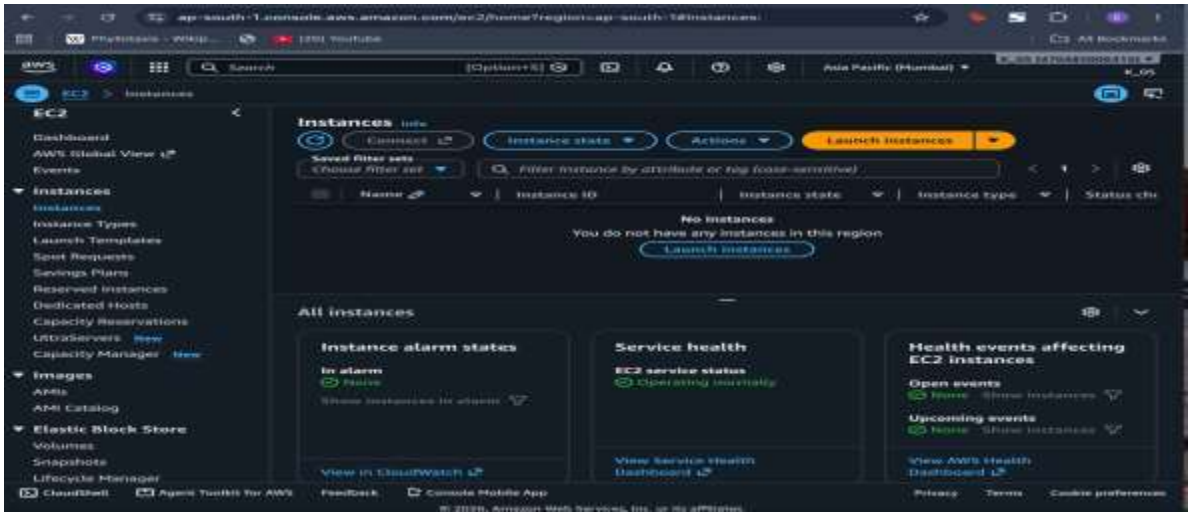
↓

terraform destroy

↓

EC2 Instance Removed

**Output:**

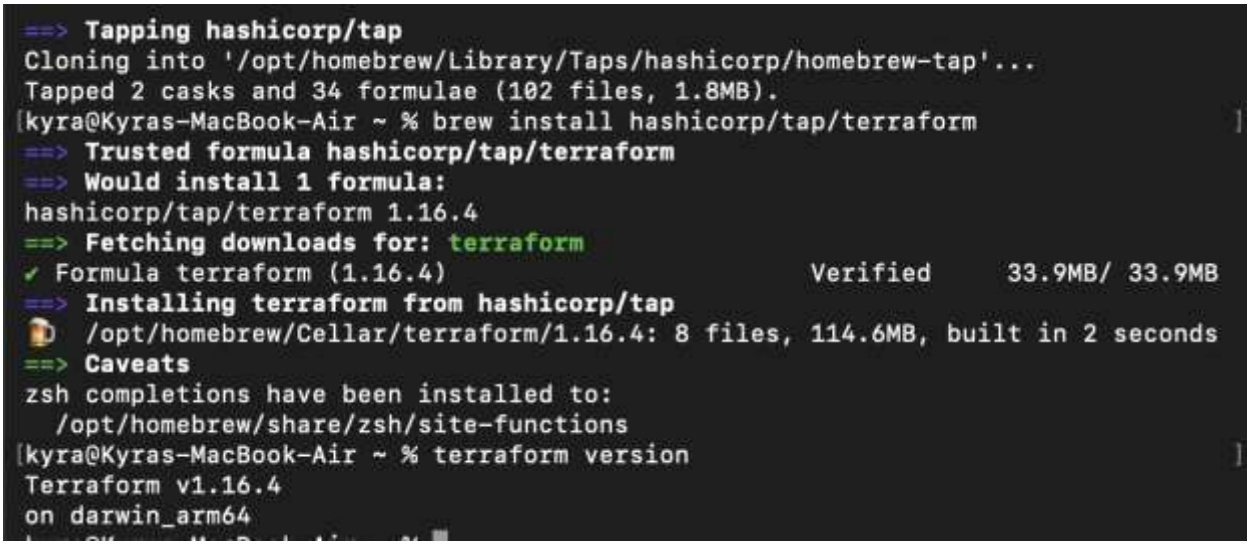


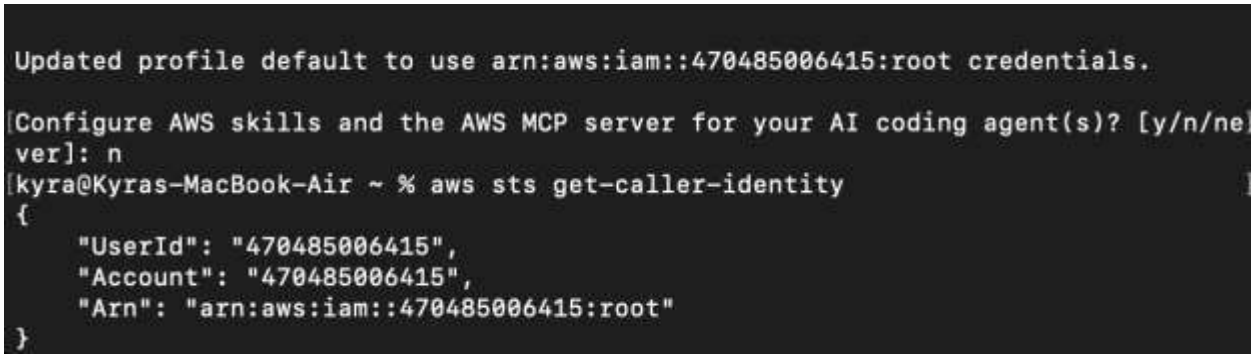

Subject : ASD&D

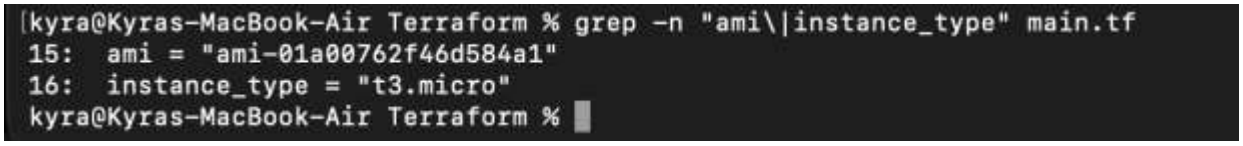
Name : Utsav Golani

Roll No : 2401032

Batch : T12



Name : Utsav Golani

Roll No : 2401032

Batch : T12

Subject : ASD&D



Name : Utsav Golani

Roll No : 2401032

Batch : T12

Subject : ASD&D

```
      + tags                                 = {
          + "Name" = "Terraform-Student-VM"
        }
      + tags_all                             = {
          + "Name" = "Terraform-Student-VM"
        }
      + tenancy                              = (known after apply)
      + user_data_base64                     = (known after apply)
      + user_data_replace_on_change          = false
      + vpc_security_group_ids               = (known after apply)

      + capacity_reservation_specification (known after apply)

      + cpu_options (known after apply)

      + ebs_block_device (known after apply)

      + enclave_options (known after apply)

      + ephemeral_block_device (known after apply)

      + instance_market_options (known after apply)

      + maintenance_options (known after apply)

      + metadata_options (known after apply)

      + network_interface (known after apply)

      + primary_network_interface (known after apply)

      + private_dns_name_options (known after apply)

      + root_block_device (known after apply)

      + secondary_network_interface (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_id   = (known after apply)
  + instance_type = "t2.micro"
  + public_ip     = (known after apply)
```

Name : Utsav Golani

Roll No : 2401032

Batch : T12

Subject : ASD&D

```
kyra@Kyras-MacBook-Air Terraform % cd ~/Terraform
kyra@Kyras-MacBook-Air Terraform % ls
main.tf         outputs.tf
kyra@Kyras-MacBook-Air Terraform % terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.66.0...
- Installed hashicorp/aws v6.66.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

```
Outputs:

instance_id = "i-0de2b5db98a8ce4bf"
instance_type = "t3.micro"
public_ip = "15.252.137.242"
kyra@Kyras-MacBook-Air Terraform % grep -n "ami\|instance_type" main.tf
15:  ami = "ami-01a00762f46d584a1"
16:  instance_type = "t3.micro"
kyra@Kyras-MacBook-Air Terraform % terraform validate
Success! The configuration is valid.

kyra@Kyras-MacBook-Air Terraform % terraform plan
aws_instance.student_vm: Refreshing state... [id=i-0de2b5db98a8ce4bf]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and
found no differences, so no changes are needed.
kyra@Kyras-MacBook-Air Terraform % terraform apply
aws_instance.student_vm: Refreshing state... [id=i-0de2b5db98a8ce4bf]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and
found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0de2b5db98a8ce4bf"
instance_type = "t3.micro"
public_ip = "15.252.137.242"
kyra@Kyras-MacBook-Air Terraform %
```

Name : Utsav Golani

Roll No : 2401032

Batch : T12

Subject : ASD&D

```
Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_id   = (known after apply)
  + public_ip     = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

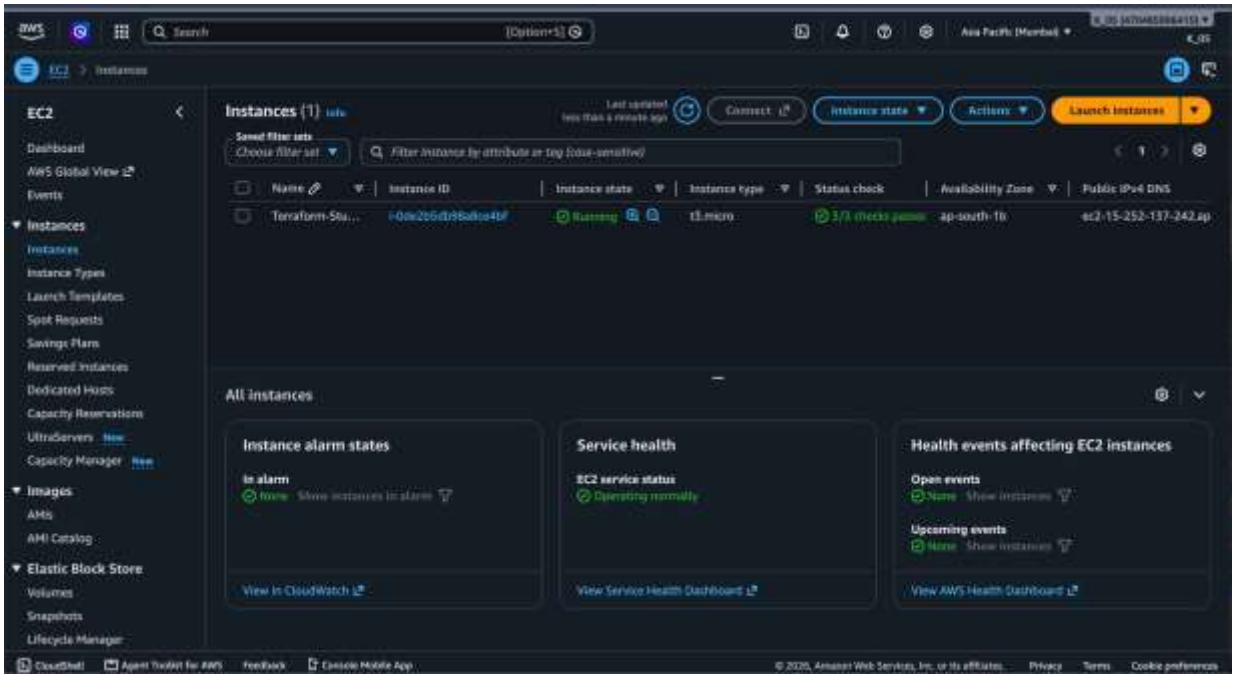
  Enter a value: yes

aws_instance.student_vm: Creating...
aws_instance.student_vm: Still creating... [00m10s elapsed]
aws_instance.student_vm: Creation complete after 13s [id=i-0de2b5db98a8ce4bf]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0de2b5db98a8ce4bf"
instance_type = "t3.micro"
public_ip = "15.252.137.242"
kyra@Kyras-MacBook-Air Terraform %
```



Name : Utsav Golani

Roll No : 2401032

Batch : T12

Subject : ASD&D

main.tf outputs.tf

main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "student_vm" {
  ami           = "ami-0018b177b79ff6b47"
  instance_type = "t2.micro"

  tags = {
    Name = "Terraform-Student-VM"
  }
}
```

**Conclusion:** In this experiment, we studied how to provision cloud infrastructure using Terraform. Terraform simplifies infrastructure management by using Infrastructure as Code (IaC), allowing cloud resources to be created, modified, and deleted automatically. It reduces manual effort, improves consistency, and makes cloud infrastructure deployment more efficient and reliable. Thus, Terraform is an important tool in DevOps and cloud computing for automated infrastructure provisioning.

LO4, LO5 and LO6 mapped successfully.